



The Unsolved Heart of Computing: P vs NP and its implication

Jay Chen



Presentation Outline

Part 1: Intro and History

Part 2: What is P vs NP?

Part 3: Implications for it's solution

Part 4: Current Progress

Intro and History

The P vs NP problem is one of seven Millennium Prize problems listed by Clays Mathematical Institution. This is an extremely difficult and important question in the field of computer science and mathematics and lies at the heart of understanding what is computationally feasible. It asks a deceptively simple question and solving it earns you \$1 million and may have massive implications for science, industry, and society.

In 1971 Stephen Cooks published a paper called “The Complexity of Theorem-proving Procedures”. Introduced the concept of NP-Completeness and formulated the problem of P vs NP. This revolutionized and defined a core part of computational complexity theory and he was awarded a Turing award (1982) for his contributions.

The problem quickly became a focal point in computer science and mathematics and is one of the hardest most important unsolved questions.



What is P vs NP?

To understand what the question asks we must first understand what P and NP are. P and NP are classes of problems known as complexity classes.

P

P refers to Polynomial time and this complexity class includes problems which have solutions in polynomial time, so $O(n^k)$.

Problems in class P have efficient algorithms where the time to compute grows reasonably with input size. In computational complexity theory Polynomial time is feasible, very good, and considered quick.

Examples of problems in class P: Finding shortest paths (Dijkstra's Algorithm), Checking if a number is prime (AKS Primality Test), Sorting a list (Merge sort, Quick sort), etc.

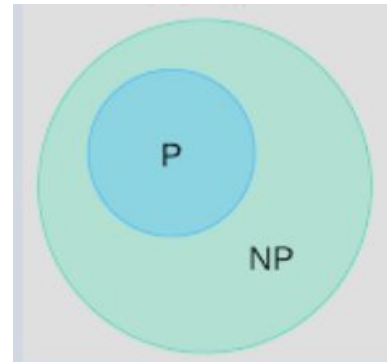
NP

NP stands for nondeterministic polynomial-time. This class of problems have solutions which can be verified in polynomial time but solving them is very hard and take a long time.

So if someone hands you a solution, you can verify it quickly (polynomial time) that it is correct, but finding the solution is not so easy.

Example of problems in class NP: Boolean Satisfiability(SAT), Travelling salesman problem, Sudoku, etc.

Note: $P \subseteq NP$, every problem in P is a also in NP, because if you can solve it fast, you can certainly verify it fast.



What does P vs NP ask?

Formally written as does $P = NP$?

It asks, is every problem that can be verified in polynomial time also solvable in polynomial time?

Informally, Is every problem whose solution can be verified quickly (NP) also one that can be solved quickly (P)?

Basically is there a quick solution for NP problems.

If the answer is yes, it would mean every problem with a quickly checkable solution also has a quickly findable one.

If no, then some problems are inherently hard to solve.

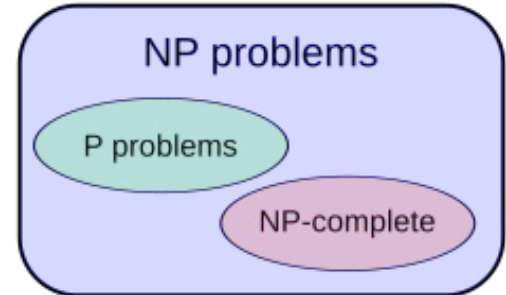
NP-completeness

NP complete is a subset of the class NP. Every problem in NP can be reduced to an NP complete problem.

In Stephen Cook's paper he proved the existence of NP-complete problems by proving that the Boolean satisfiability (SAT) problem is NP-complete. He showed that every problem in NP can be translated (reduced) to SAT in polynomial time.

In 1972 Richard Karp expanded on Cook's work by listing 21 NP-complete problems.

Examples include: SAT, Traveling Salesmen, Vertex cover, etc.



Polynomial Time Reduction

Cooks formalized and introduced polynomial time reduction as a method for solving one problem using another, by reducing one NP problem into an NP complete problem.

Let's say you don't know how to solve problem A but I know how to solve and convert it into problem B, solve that, convert the solution back.

This property of NP completeness is important because if any problem in NP can be converted to a problem in NP-Complete, this would mean if you find a solution to even one NP-complete problem in polynomial time you have found a solution for every problem in NP in polynomial time. Thereby proving that yes, $P = NP$!

NP complete problems are the main battleground when trying to solve P vs NP .

Implications

If $P = NP$ how does it impact the world?

As you can imagine, algorithms is used in nearly every field. A quick way to solve hard problems will revolutionize industries, science, and society in good and bad ways.

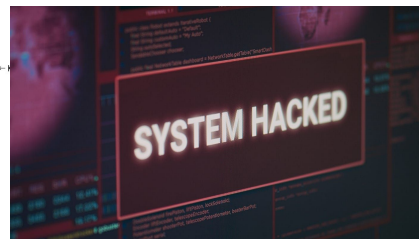
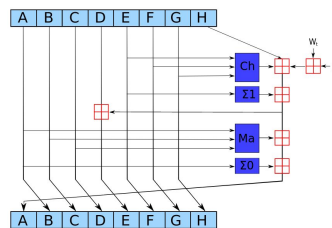
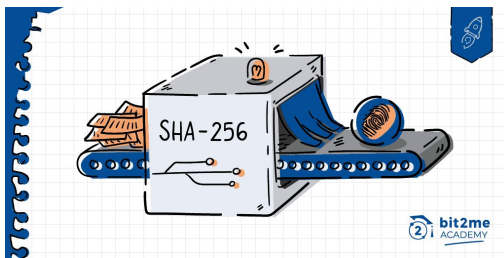
The Collapse of Cryptographic Security

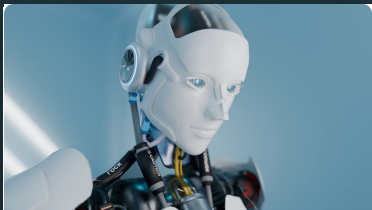
Most all cryptographic systems rely on the difficulty and hardness of NP problems, they use math problems that are hard to solve but easy to verify.

Some Cryptographic protocols

- Hash functions like Sha-256 which is used in blockchain (Bitcoin mining), based on difficulty of inverting the hash function and difficulty of finding collisions, an NP problem.
- RSA (Rivest-Shamir-Adleman) which is used for secure web browsing (HTTPS), Encrypted emails, and digital signatures is based on the difficulty of factoring large composite integers into primes which is an NP problem.
- ECC (Elliptic Curve Cryptography) which is used for Cryptographic wallets (bitcoin, Ethereum) and Secure messaging is based on the Elliptic curve discrete logarithm problem which is again NP.

If $P = NP$, then what was once unfeasible suddenly becomes plausible. What once took 10,000 years of brute-force search, could now be done in 10 seconds.





AI / Machine Learning

Many bottlenecks in AI are combinatorial and NP-hard for example Constraint Satisfaction problems (scheduling, planning), neural architecture search, and probabilistic inference.

These problems could be solved efficiently and exactly, AI could grow in magnitude as compared to the previous years. Natural language processing, image recognition, and decision-making tasks that now require massive computing power or heuristics could be solved in seconds. Entire neural networks could be optimized automatically and instantly.

Biology & Chemistry

Problems like protein folding (how a chain of amino acids folds into a 3D structure) are NP-hard. Drug design involves searching enormous chemical spaces.

If $P = NP$, exact protein folding could be solved instantly, drug candidates could be designed in hours instead of years, Genome alignment, evolutionary modeling, and synthetic biology design become faster and more accurate



Mathematics

Automated theorem proving, a historically hard task, would become efficient. Proofs that once took years of human effort could be found by machines in minutes. Could generate new theorems and proofs without human input. Mathematics could become machine-driven, producing theorems and proofs far faster than human mathematicians.

Logistics & Operations

Industries like transportation, manufacturing, and supply chain management rely on solving NP-hard optimization problems such as the Traveling Salesman Problem, knapsack, and job scheduling. If $P = NP$, these could be solved optimally in real time, making global workflows faster, cheaper, and more efficient — saving billions of dollars in the process.

If $P \neq NP$

- Some problems are just fundamentally hard.
- Reassurance that our systems stay secure.
- Further motivate research and innovation in approximation algorithms, heuristics, quantum computing.

Current Progress

- Question is still unsolved, there has been no proof that $P = NP$ or $P \neq NP$.
- Thousands of problems proven NP-complete (via polynomial reductions).
- Most experts believe $P \neq NP$, but proving it has turned out to be incredibly difficult.
- Researchers encountered a roadblock called the natural proofs barrier, If natural proofs could separate P from NP, they could also be used to break encryption, specifically, by breaking pseudorandom generators, which are the foundation of many cryptographic systems. That's why they're unlikely to solve P vs NP.
- When asked whether we need a new theoretical approach to separate complexity classes and solve P vs NP problems? Stephen Cook says in an interview "the sad story is we just don't know how to proceed to do it".